🚀 Lecture 04.06: List Iteration & Comprehension Paradigms

📌 Executive Architectural Summary

Iterating over lists is fundamental to feature processing pipelines. Python offers direct iterable traversal, indexed enumeration via `enumerate()`, parallel iteration via `zip()`, and declarative **List Comprehensions** executed at C speed.

🔬 Core Engineering Mechanics

1. The Iterator Protocol

A list implements `__iter__()`, which returns a `list_iterator` object yielding pointers via `__next__()` until raising `StopIteration`.

2. List Comprehensions vs For Loops

List comprehension syntax:

$$[f(x) ext{ for } x \in L ext{ if } P(x)]$$

List comprehensions execute faster than equivalent explicit `for` loops because loop mechanics and append calls are optimized directly in CPython's bytecode (`LIST_APPEND`).

💻 Production Python Implementation

```
# 1. Direct Traversal with enumerate()
languages = ["Python", "C++", "Rust", "Julia"]
for idx, lang in enumerate(languages, start=1):
    print(f"Rank {idx}: {lang}")

# 2. Parallel Traversal with zip()
names = ["Alice", "Bob", "Charlie"]
scores = [92, 88, 95]
for name, score in zip(names, scores):
```

```
    print(f"Student: {name} -> Score: {score}")

# 3. List Comprehensions
raw_numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even_squares = [x ** 2 for x in raw_numbers if x % 2 == 0]
print("Even Squares Comprehension:", even_squares)
```